Carnegie Mellon University
Master of
Software Engineering

17-647 Assignment A3: Circuit Breaker, Kubernetes (EKS), Kafka, DB per ms

Problem Description

In this project assignment, we will introduce additional concepts and technologies discussed in class:

- Move the containers to Kubernetes (K8S) using Amazon's Elastic Kubernetes Service (EKS).
- Use a K8S liveness probe to monitor services.
- Book service will call an external recommendation engine (external service that will be provided).
- Create a timeout and circuit breaker for the interaction with the external service.
- Define an asynchronous messaging interaction using Kafka (Kafka cluster will be provided) and create a Kafka consumer (asynchronous service) that will send an email to newly registered customers.
- Apply the DB per microservice pattern to Book and Customer services.

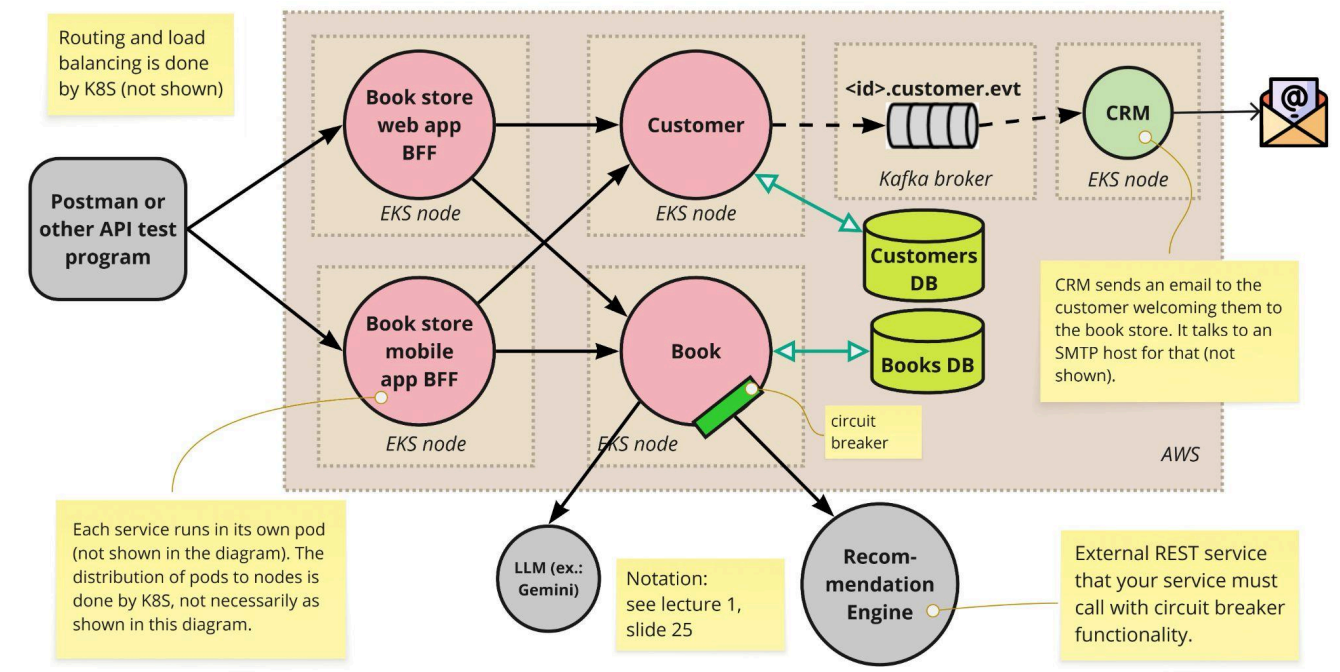
Learning Objectives

This project is designed to introduce you to core technologies and design strategies and patterns commonly used in distributed systems. These technologies and design strategies include:

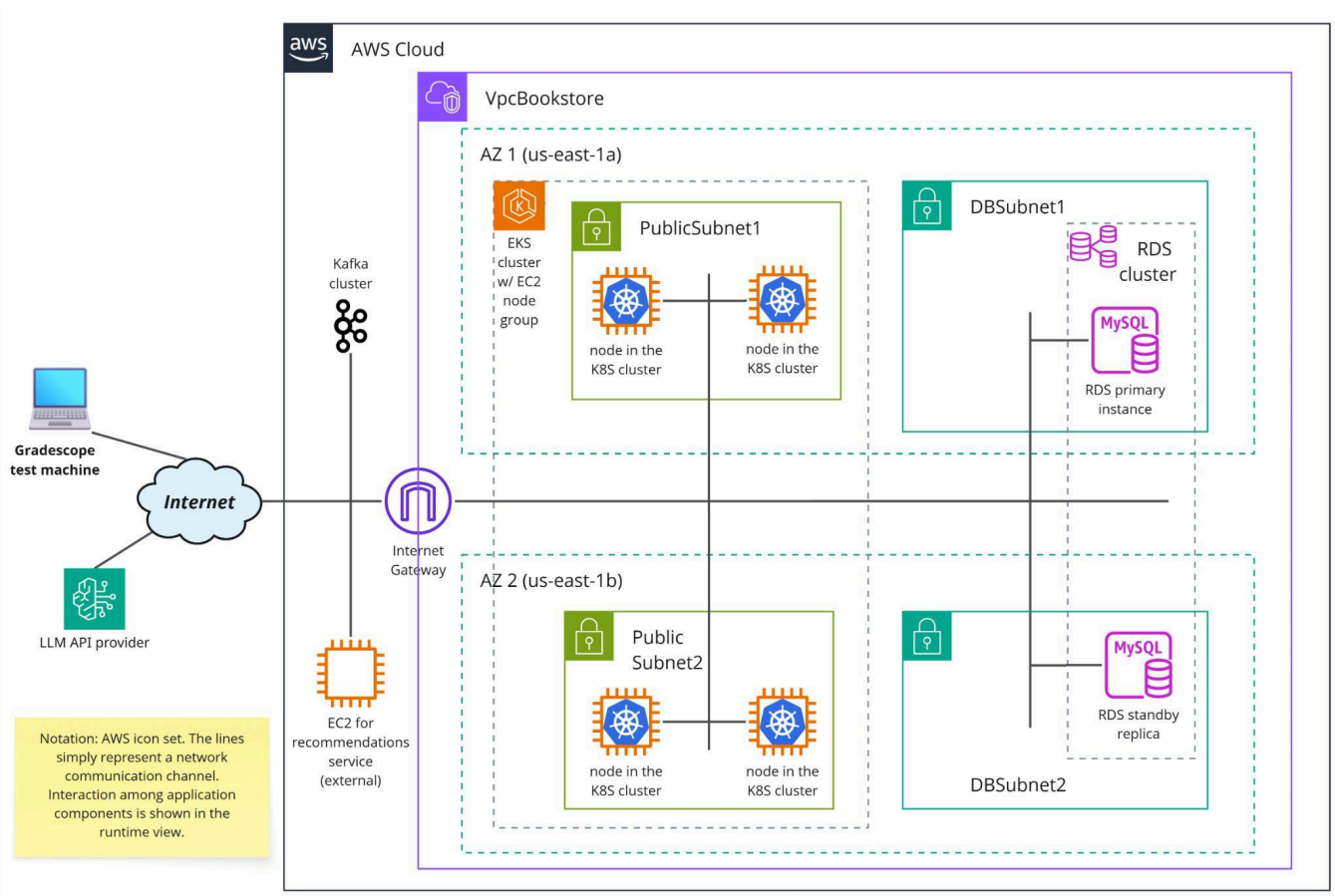
- AWS services: EC2, VPC, CloudFormation (CF)
- MySQL on AWS RDS
- Load balancer and passive redundancy
- Postman (or similar API testing tool)
- Docker
- Python with Flask, Node JS and Express or other language/framework that you choose to create your services
- Service-to-service HTTP interaction
- JWT (added in A2)
- BFF pattern (added in A2)
- Single Responsibility Principle (added in A2)
- Kubernetes and EKS (added in A3)
- Kafka and publish-subscribe (added in A3)
- Circuit breaker and bulkheads (added in A3)
- CloudWatch for log consolidation and monitoring (added in A3)
- DB per microservice pattern (added in A3)

Architecture overview

The diagram below shows the **runtime architecture** perspective that you should implement for this assignment. The details are in the following sections.



The diagram below shows the **AWS deployment** perspective. The diagram uses the standard AWS notation for AWS infrastructure design diagrams (the K8S and Kafka logos are not part of that).



Task 1: deploy on EKS

In this assignment, you take the services that you developed in A2 and **deploy them on a Kubernetes cluster** managed by AWS (EKS).
An EKS cluster contains a number of **nodes**. Each node is an EC2 instance that the EKS service *automatically* allocates to the cluster based on a **node group** configuration. Therefore, we will no longer need to deal with EC2 instances directly for deploying the application services. Yay! 😀 We're moving to a full-fledged PaaS service (EKS), leaving behind an IaaS service (EC2).
K8S is a container orchestration platform, so we won't have to docker pull/run/stop images/containers, as K8S will do that for us. Also, **pods** become the directly visible execution elements. You will still build and push docker images to a registry; EKS will pull your images and run them inside pods in the cluster. (It's still advisable to run docker images on your local dev environment though.)

You'll need to create an EKS cluster, which can be done via the provided CF template mentioned further below.
Then you need to define K8S [configuration files]() for each service:

- `kind: Deployment` ⇒ *defines the pod properties, such as number of replicas, container images to run in the pod (in our case it's one image per pod), and liveness probe.*
- `kind: Service` ⇒ *basic configuration to make the service in the pod visible on the network through a port.*

In the K8S deployment file, you shall specify the number of instances (`replicas`) of each pod:

- 2 pod instances for Book store web app BFF.
- 2 pod instances for Book store mobile app BFF.

- 2 pod instances for Customer service.
- 1 pod instance ~~2 pod instances~~ for Book service (reason for 1 instance explained further below).
- 1 pod instance for CRM (new, asynchronous service explained further below)

All application services should be in the same K8S namespace named `bookstore-ns`. You'll need to create a K8S configuration file for the namespace as well.

All your K8S configuration files must be written in yaml. Each separate K8S deployment is a different microservice. Your code repo should be organized to reflect that: the implementation units and K8S yaml files (Deployment, Service) should be separate for each microservice. Hint: in your K8S deployment files, use `imagePullPolicy: Always` to make sure K8S always pulls the docker image from the registry (otherwise it might not if it has already pulled that image with the same tag).

You will monitor the health of the REST services with EKS. In your K8S deployment file, you shall specify a liveness probe to point to the `GET /status` endpoint (as specified in previous assignments). All **REST services** shall have the liveness probe configured as such.

> We typically configure routing on K8S in a yaml file of type *Ingress*. The Ingress points to the *Service* and the Service points to the *Deployment* (via the app label selector). The Ingress file also defines a *host* (example: pmerson-ediss-bookstore.amazonaws.com). However, the Learner Lab AWS environment does not allow the specification/registration of a host+domain name. In other words, the traditional mechanism of routing requests via Ingress apparently is not possible on LL. Remember, in A2 we used the external ALB to route requests to the BFFs based on the X-Client-Type header. In A3, we need to make the Web app BFF and the Mobile app BFF available on different locations. For that, we use a Service of type *LoadBalancer*. It's also the reason why you must submit two URLs, one for each BFF service.

Task 2: new endpoint for "related books"

Your system will continue to support all the API endpoints, as specified in A2. In addition, you'll support the "related books" endpoint specified below.

Retrieve Related Books endpoint

- `GET {BASEURL}/books/{ISBN}/related-books`
- **Description:** returns a list of book titles that are related to a given book specified via its ISBN.
- **Return Values:**
  - Success with at least one book in the response.
    - HTTP Code: 200
    - Response Body (example):

```
[
    {
        "ISBN": "978-0321815736",
        "title": "Software Architecture in Practice",
        "Author": "Bass, L."
    },
    {
        "ISBN": "978-0-321-55268-6",
        "title": "Documenting Software Architectures Second Edition",
        "Author": "Clements, P. et al"
    }
]
```

  - Successful execution, but no recommended books found.
    - HTTP Code: 204
    - Response Body is empty.
  - Call to external service times out (details below)

    - HTTP Code: 504
  - The circuit is open in the circuit breaker (details below)
    - HTTP Code: 503

Your system will access an **external** recommendation engine service every time the "related books" endpoint is executed and return recommendations for additional books the customer may want to purchase. *The URL of this external service to be used for grading will be announced on Canvas, but you can run and see the API documentation locally (instructions are further below).* The external service has an endpoint that takes the ISBN as a parameter. The ISBNs you can use that will return a non-empty response are:

- 978-0-321-55268-6
- 978-0321815736
- 9780133065107
- 978-0395489321
- 978-0544174221

Task 3: circuit breaker

Circuit breaker for the interaction with the recommendation engine:

- The call to the external service shall not significantly delay the response to the end user (simulated by the autograder).
- Thus, the Book service shall get a response from the recommendation engine within **3 seconds** of the request being sent. (Different frameworks/libraries for creating HTTP client code typically provide an option to specify the timeout for requests.)
- In order to support that requirement, you should implement a *circuit breaker* that will open if the external service is slow or unresponsive.
- For simplicity, you can move the circuit to the *open* state upon the first call that times out.
- When the call to the external service times out, *your service* should respond with error 504.
- When the circuit is open, your service should automatically return error 503 right away to any requests within 60 seconds of opening the circuit.
- The first request that arrives at the Book service after 60 seconds elapsed since the circuit was open will be different:
  - Your service should try to reach the external service.
  - If it responds within 3 seconds, then you move the circuit back to the *closed* state, and your service's response code is 200.
  - If it does not respond within 3 seconds, your service's response code is 503, the circuit remains open, and you start another 60-second window until the next attempt.
- The circuit breaking logic is only for `GET /books/{ISBN}/related-books`
- The circuit breaking functionality requires keeping state across calls. The ideal solution should not violate the *stateless* architecture constraint of the REST style. For simplicity, the circuit breaker state (open | closed) may be visible only within a given pod[1].
  - One way to do that is to store the state to disk on a *K8S volume* that is mounted by the Book service pod. You can use the `emptyDir` volume type.
  - Because of this limitation and to avoid non-deterministic behavior *during the autograder tests*, there should be only one instance of the Book service.
- You can assume that if one call to the external service times out, any call to the external service will time out (regardless of the ISBN value).

For your own tests, you can deploy and run a version of this external recommendation engine service in your AWS space or local machine. The docker image is available on docker hub:

[1] A more sophisticated solution would make the state visible to all instances of the Book service, that is, if one instance (one pod) changes the state to open, the other instances can see that state change. But we will not go that far in this assignment because it would require mounting a volume with cross-pod visibility (or similar solution).

```
docker pull pmerson/book-recommendations-ms
```

To run, use this command:

```
docker run -d -p 80:8080 pmerson/book-recommendations-ms --delay=5000
```

The *delay* argument is how many milliseconds the service will sleep for when processing any request.
Once it's running you can see the API documentation on the browser at `<base-url>/swagger-ui.html`

When you turn in your solution, your service should be calling the external service on the URL that will be later announced on canvas, not the URL you used during development.

How to test your circuit breaker

The Book service shall have a 3-second timeout when calling the recommendation service. When you launch the recommendation service, `--delay` is a command line argument that will allow you to test your circuit breaker. Example of test scenario:

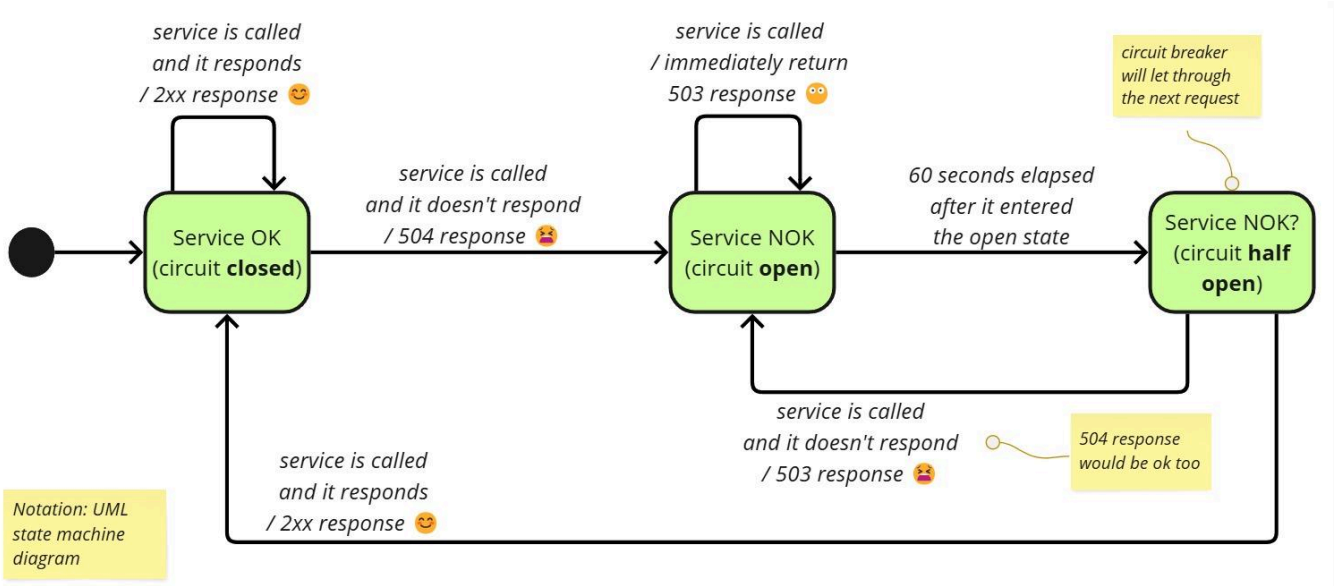
1) `docker run ... --delay=0`
2) you call `GET /books/{ISBN}/related-books` on the Book BFF service ⇒ the external recommendation service is responding with no delay, so the circuit should be **closed**, and the "end user" should get the response and status code 200 or 204.
3) `docker stop ...`
4) `docker run ... --delay=10000`
5) you call `GET /books/{ISBN}/related-books` ⇒ the call to the recommendation service should time out; the "end user" should get 504 and the circuit should move to **open**.
6) you quickly (in less than 60 seconds) call again `GET /books/{ISBN}/related-books` ⇒ the circuit is **open**, so the "end user" should get 503 right away.
7) after more than 60 seconds you call `GET /books/{ISBN}/related-books` ⇒ the circuit is **open**[2], but the Book service tries again to reach the recommendation service. It's still running with `--delay=10000`, so the call will time out, the "end user" should get 503, and the circuit should remain **open**.
8) you quickly (in less than 60 seconds) call again `GET /books/{ISBN}/related-books` ⇒ the circuit remains **open**, so the "end user" should get 503 right away.
9) `docker stop ...`
10) `docker run ... --delay=0`
11) you quickly (in less than 60 seconds after step 8) call again `GET /books/{ISBN}/related-books` ⇒ the circuit remains **open**, so the "end user" should get 503 right away.

[2] Strictly speaking, this would be the half-open state, as seen in the state machine diagram.

12) after more than 60 seconds you again call `GET /books/{ISBN}/related-books` ⇒ the circuit is **open**, but it's time to try once more to reach the recommendation service. Now it's running with `--delay=0`, so the call succeeds, the "end user" gets 200 or 204, and the circuit moves to the **closed** state.

13) you call `GET /books/{ISBN}/related-books` ⇒ the service responds with no delay, the "end user" gets 200 or 204, the circuit remains **closed**, and your application with circuit breaker functionality is working. Yay! Good job!

Below is a state machine diagram that illustrates the behavior of the circuit breaker as specified above.



Task 4: asynchronous service with Kafka

There is an additional requirement to the `POST /customers` endpoint. If the request to create a customer is valid, the Customer service must send a message to a topic named `<your-andrew-ID>.customer.evt` on Kafka. The address of the Kafka brokers will be provided in an announcement on Canvas. The individual Kafka topics will already exist on these brokers–you don't need to create any Kafka topic. The message should be in JSON format and contain the same attributes used in the REST service response. This message corresponds to the *Customer Registered* domain event.

You also need to create an asynchronous service (CRM in the runtime architecture diagram). It is a Kafka consumer that shall connect to the Kafka topic mentioned above. Upon receiving the message, the CRM service shall parse the content and send an email to the newly registered customer. The subject of the email must be: "Activate your book store account". The body of the email should say

> Dear <customer name>,
> Welcome to the Book store created by <your andrew ID>.
> Exceptionally this time we won't ask you to click a link to activate your account.

To send emails your CRM service needs to communicate with an external SMTP host. You should use an email library for your chosen development platform for that.

Examples of options for the SMTP host:

- If you have a gmail account, you can use Gmail app authorization.
  - If you follow the instructions and yet don't see the App passwords option at the bottom of the page, you can simply go to https://myaccount.google.com/apppasswords
  - If you don't want to use your personal gmail account, you can create a new one (e.g., ediss_myandrewid@gmail.com) and enable app passwords there.
  - App passwords are not enabled for andrew email accounts.
  - Example of config (application.yaml snippet in a Spring Boot app):

```
spring:
 mail:
   host: smtp.gmail.com
   port: 587
   username: myaccount@gmail.com
   password: "lwmr bnon hqpa ryfw"
   properties:
     mail:
       smtp:
         auth: true
         starttls:
           enable: true
```

- Amazon has the Simple Email Service (SES), but people had trouble trying to use it in LearnerLab in the past.
- Other email providers (e.g., yahoo mail) may also give you the ability to send SMTP messages from your app or 3rd party email client. However, many of them are closing this option for security reasons.

The CRM service must be dockerized and deployed to the EKS cluster using the same mechanisms used for the REST services. However, the CRM K8S deployment does not need to specify a liveness probe. You can configure the CRM service K8S deployment to 1 instance (replica).

Task 5: DB per microservice pattern

The A3 solution shall remain using the RDS MySQL database cluster you created for A1 and A2 on AWS. However, there should be two separate databases on the MySQL cluster, one for books and the other for customers. The Customer service is the only application component that can connect to the Customers DB. The Book service is the only component that can connect to the Books DB.

You are expected to use good programming practices with proper error handling.

You will also need to be sure to comply *exactly* with the specification provided. Any deviation will result in a failed test case even if the functionality is implemented correctly.

Technologies and Resources

In this assignment, you should also configure the AWS infrastructure and deploy your application using our Learner Lab course in the AWS Academy platform. To set up the infrastructure, you should download and use the CF-A3-cmu.yml template from this github repo. *This template is a starting point. You should edit it or manually create resources on the AWS console as you see necessary for your solution.*

Please refer to tutorials and links in previous assignments. Here are some additional links that can help you with the technologies introduced in A3.

- Install the AWS CLI
- Add self-managed nodes to an EKS cluster
- Deploy a sample application - Amazon EKS
- Mounting an emptyDir K8S volume: https://cloud.google.com/kubernetes-engine/docs/how-to/volumes

Additional Tutorials:

- Explanation of K8S ingress and service.

Getting Started

In A3 you need to interact with K8S/EKS. There are three basic approaches to interact with this kind of infrastructure:

1) Using a command line interface (CLI). The basic CLI tools to use are `kubectl` and `eksctl`.
2) Using a web console provided by the vendor of the container orchestration platform.
3) Creating scripts that use an Infrastructure as Code (IaC) tool, such as terraform and kustomize.

In these assignments, the recommended approach is (1). It takes some effort to learn the CLI arguments, but you can get help from your favorite GenAI and also on Piazza if necessary.
As a suggestion, keep a journal on a text file with all the CLI commands you use to set up, validate, configure, and shutdown resources. Add comments so you know what each command is used for.

Submission

- For this project, you'll submit a zip file or git repo[3] of the implementation artifacts (source code, Dockerfile, config files) to Gradescope. Ensure that your project directory contains only the necessary implementation files. Remove any unnecessary files or directories, such as "node_modules" for Node.js projects, build artifacts, and temporary files.
- There are 5 microservices. They should be clearly separated in subdirectories.
- The K8S yaml configuration files must be submitted along with the other implementation files.
- url.txt:
    - The file should contain four lines in the following format:
      ```
      {base_url_web_app_bff}
      {base_url_mobile_app_bff}
      {andrew_id}
      {email_address}
      ```
    - The `{email_address}` is the one used by the CRM service to send emails.
    - Example:
      ```
      http://a030ae7020a204eee94ac233e7252e09-583638513.us-east-1.elb.amazonaws.com:80
      http://a1e5b15950d6a40e99f92efeba3788b0-1709868825.us-east-1.elb.amazonaws.com:80
      johndoe
      johndoe@gmail.com
      ```

Grading

The grading for this project will be:

- The requirements were correctly implemented.

[3] Some people have had problems with zip file submission even after removing unnecessary files. If that happens, please submit a github monorepo (single git repo containing multiple components in separate directories).

- The system is able to handle correct and invalid input and return the correct HTTP status code in the response.
- The architecture of the system matches the instructions provided.